1. Single Inheritance

```
class Person{
    String name="Bhavya";
    int age=19;
}
class Student extends Person{
    int rollno=101;
    String branch="AIML";
    void display(){
        System.out.println("Name: "+super.name);
        System.out.println("Age: "+super.age);
        System.out.println("Roll No: "+rollno);
        System.out.println("Branch: "+branch);
    }
}
public class SInherit{
    public static void main(String[] args){
        Student s1=new Student();
        s1.display();
    }
}
```

Output:

Name: Bhavya

Age: 19

Roll No: 101

Branch: AIML

2. Using super Keyword

```
3. class Vehicle{
4.     int speed = 150;
5.     void display(){
6.         System.out.println("Speed(Vehicle): "+speed);
7.     }
8. }
9. class Car extends Vehicle{
10.    String model = "Porsche";
```

```
11.     @Override
12.     void display(){
13.         super.display();
14.         System.out.println("Speed("+model+"): "+super.speed);
15.     }
16.}
17.public class Super{
18.     public static void main(String[] args){
19.         Car c = new Car();
20.         c.display();
21.     }
22.}
```

Output:

Speed(Vehicle): 150

Speed(Porsche): 150

3. Multilevel Inheritance

```
class Person{
    String name;
    int age;
    public Person(String name,int age){
        this.name=name;
        this.age=age;
    }
    public void displayP(){
        System.out.println("Name: "+name);
        System.out.println("Age: "+age);
    }
}
class Employee extends Person{
    int empid;
    double sal;
    public Employee(String name,int age,int empid,double sal){
        super(name,age);
        this.empid=empid;
        this.sal=sal;
    }
    public void displayE(){
        displayP();
        System.out.println("Employee ID: "+empid);
```

```
        System.out.println("Salary: "+sal);
    }
}
class Manager extends Employee{
    String dept;
    public Manager(String name,int age,int empid,double sal,String dept){
        super(name,age,empid,sal);
        this.dept=dept;
    }
    public void displayM(){
        displayE();
        System.out.println("Department: "+dept);
    }
}
public class MultiInherit{
    public static void main(String[] args){
        Manager m = new Manager("Ravi",27,102,50000,"Tech");
        m.displayM();
    }
}
```

Output:

Name: Ravi

Age: 27

Employee ID: 102

Salary: 50000.0

Department: Tech

4. Method Overriding

```
class Animal {
    void sound() {
        System.out.println("Animal makes a sound");
    }
}
class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Dog barks");
    }
```

```
}
class Cat extends Animal {
    @Override
    void sound() {
        System.out.println("Cat meows");
    }
}
public class MethodOverride {
    public static void main(String[] args) {
        Animal a1 = new Dog();
        Animal a2 = new Cat();

        a1.sound();
        a2.sound();
    }
}
```

Output:

Dog barks

Cat meows

5. Dynamic Method Dispatch

```
class Shape {
    void draw() {
        System.out.println("Drawing a shape");
    }
}
class Circle extends Shape {
    @Override
    void draw() {
        System.out.println("Drawing a Circle");
    }
}
class Rectangle extends Shape {
    @Override
    void draw() {
        System.out.println("Drawing a Rectangle");
    }
}
class Triangle extends Shape {
    @Override
```

```
    void draw() {
        System.out.println("Drawing a Triangle");
    }
}
public class Dynamic {
    public static void main(String[] args) {
        Shape shape;
        shape = new Circle();
        shape.draw();
        shape = new Rectangle();
        shape.draw();
        shape = new Triangle();
        shape.draw();
    }
}
```

Output:

Drawing a Circle

Drawing a Rectangle

Drawing a Triangle

6. super with Method Overriding

```
class Employee {
    void display() {
        System.out.println("Employee details");
    }
}
class Manager extends Employee {
    @Override
    void display() {
        super.display();
        System.out.println("Manager details");
    }
}
public class SuperMethodOverride {
    public static void main(String[] args) {
        Manager m = new Manager();
        m.display();
    }
}
```

Output:

Employee details

Manager details

7. Multilevel Inheritance for Salary Calculation

```
class Employee {
    double basicSalary = 30000;
}
class Developer extends Employee {
    double programmingAllowance = 5000;
}
class SeniorDeveloper extends Developer {
    double projectAllowance = 8000;
    void displaySalary() {
        double totalSalary = basicSalary + programmingAllowance +
projectAllowance;
        System.out.println("Basic Salary: " + basicSalary);
        System.out.println("Programming Allowance: " + programmingAllowance);
        System.out.println("Project Allowance: " + projectAllowance);
        System.out.println("Total Salary: " + totalSalary);
    }
}
public class MultiInheritSal{
    public static void main(String[] args) {
        SeniorDeveloper sd = new SeniorDeveloper();
        sd.displaySalary();
    }
}
```

Output:

Basic Salary: 30000.0

Programming Allowance: 5000.0

Project Allowance: 8000.0

Total Salary: 43000.0

8. Dynamic Method Dispatch for Bank Accounts

```
class BankAccount {
    void calculateInterest() {
        System.out.println("Calculating bank account interest");
    }
}
class SavingsAccount extends BankAccount {
    @Override
    void calculateInterest() {
        System.out.println("Savings Account Interest: 5%");
    }
}
class CurrentAccount extends BankAccount {
    @Override
    void calculateInterest() {
        System.out.println("Current Account Interest: 3%");
    }
}
public class DynamicBankAcc {
    public static void main(String[] args) {
        BankAccount account;
        account = new SavingsAccount();
        account.calculateInterest();
        account = new CurrentAccount();
        account.calculateInterest();
    }
}
```

Output:

Savings Account Interest: 5%

Current Account Interest: 3%

9. Constructor Execution in Multilevel Inheritance

```
class Person {
    Person() {
        System.out.println("Person constructor executed");
    }
```

```
}
class Student extends Person {
    Student() {
        System.out.println("Student constructor executed");
    }
}
class GraduateStudent extends Student {
    GraduateStudent() {
        System.out.println("GraduateStudent constructor executed");
    }
}
public class MultiInheritCons {
    public static void main(String[] args) {
        GraduateStudent gs = new GraduateStudent();
    }
}
```

Output:

Person constructor executed

Student constructor executed

GraduateStudent constructor executed

10. Banking Application Using Inheritance

```
class BankAccount {
    int accountNumber;
    double balance;
    BankAccount(int accountNumber, double balance) {
        this.accountNumber = accountNumber;
        this.balance = balance;
    }
    void deposit(double amount) {
        balance += amount;
        System.out.println("Deposited: " + amount);
    }
    void withdraw(double amount) {
        if (amount <= balance) {
            balance -= amount;
            System.out.println("Withdrawn: " + amount);
        } else {
```

```
            System.out.println("Insufficient balance");
        }
    }
    void calculateInterest() {
        System.out.println("Interest calculation for Bank Account");
    }
    void displayBalance() {
        System.out.println("Account Number: " + accountNumber);
        System.out.println("Balance: " + balance);
    }
}
class SavingsAccount extends BankAccount {
    SavingsAccount(int accountNumber, double balance) {
        super(accountNumber, balance);
    }
    @Override
    void calculateInterest() {
        double interest = balance * 0.05;
        System.out.println("Savings Account Interest: " + interest);
    }
}
class CurrentAccount extends BankAccount {
    CurrentAccount(int accountNumber, double balance) {
        super(accountNumber, balance);
    }
    @Override
    void calculateInterest() {
        double interest = balance * 0.03;
        System.out.println("Current Account Interest: " + interest);
    }
}
public class BankAppInherit {
    public static void main(String[] args) {
        SavingsAccount savings = new SavingsAccount(101, 10000);
        System.out.println("Savings Account:");
        savings.deposit(2000);
        savings.withdraw(1000);
        savings.calculateInterest();
        savings.displayBalance();
        System.out.println();
        CurrentAccount current = new CurrentAccount(102, 20000);
        System.out.println("Current Account:");
        current.deposit(5000);
        current.withdraw(3000);
        current.calculateInterest();
```

```
        current.displayBalance();
    }
}
```

Output:

Savings Account:

Deposited: 2000.0

Withdrawn: 1000.0

Savings Account Interest: 550.0

Account Number: 101

Balance: 11000.0

Current Account:

Deposited: 5000.0

Withdrawn: 3000.0

Current Account Interest: 660.0

Account Number: 102

Balance: 22000.0